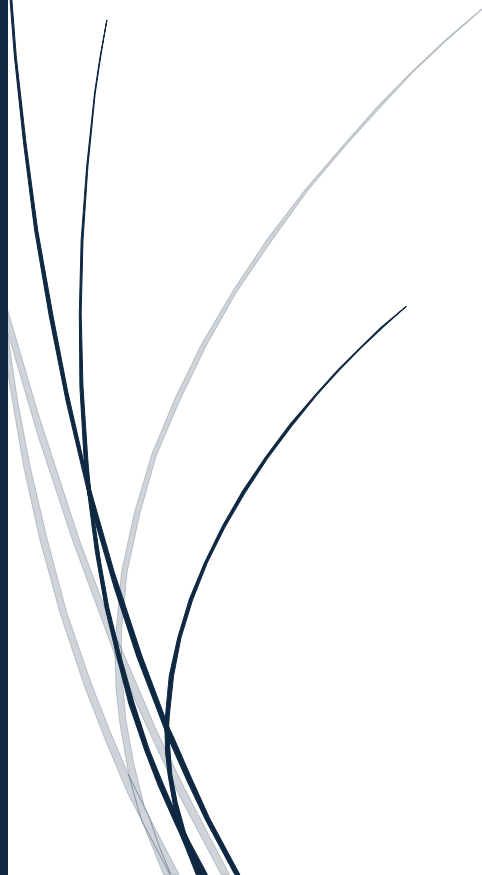




Entregable n2

CROOSFITBOX

Curso SQL - Coderhouse



Emiliana María Torres
CODERHOUSE

INFORME

Introducción

En este documento se presenta la información correspondiente a la base de datos diseñada para su implementación en un gimnasio o box de CrossFit. En ella se gestionan clases de Crossfit, entrenamiento Funcional y Weightlifting. Además, se incluye un análisis descriptivo con una tabla que muestra el contenido detallado de la base de datos, junto con un diagrama esquemático que representa las relaciones entre sus entidades.

Situación problemática

En un gimnasio o box especializado en CrossFit, Funcional y Weightlifting, la gestión de forma manual de las inscripciones, reservas de clases, asignación de entrenadores y el control de pagos genera demoras, errores y falta de seguimiento. Por lo tanto, esta situación dificulta tanto el trabajo del personal como la experiencia del cliente.

Por ello, surge la necesidad de diseñar una base de datos que permita centralizar la información, mantener un registro confiable de las operaciones diarias y optimizar la administración del servicio, evitando inconvenientes que puedan surgir por la falta de control o la duplicación de información.

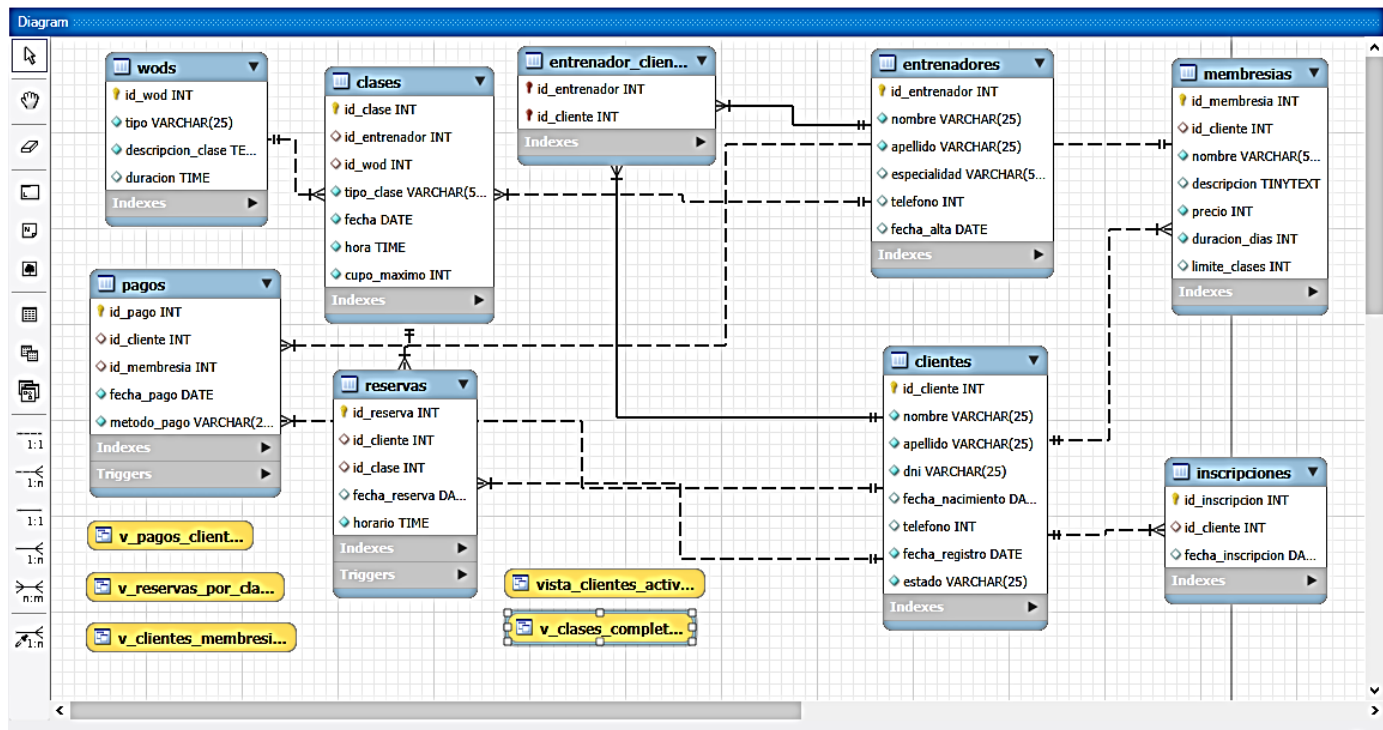
Objetivos

El objetivo principal es mejorar la performance del manejo del box, con el fin de tener todo organizado y plasmado en una base de datos.

Implementar una base de datos que ordene y automatice los procesos operativos del box, facilitando la gestión de clases, entrenadores y pagos.

Tener un sistema estructurado y eficiente para mejorar el rendimiento administrativo, asegurar un seguimiento detallado de las actividades y garantizar una experiencia satisfactoria tanto para los clientes como para el personal del gimnasio.

Diagrama Entidad Relación (DER)



VISTAS

1. Vista_clientes_activos

El objetivo de esta vista es ver rápidamente los clientes que están habilitados. Nos muestra:
Id cliente, nombre, apellido, dni, fecha registro.

2. Vista_clases_completas

El objetivo es mostrar toda la info relevante de una clase, nos puede servir como una agenda diaria. Unimos las tablas clases, entrenadores y wods con un JOIN.

De la tabla clases usamos las columnas id_clase, fecha, hora, tipo_clase y cupo_maximo.

De la tabla entrenadores usamos las columnas nombre como nombre_entrenador y apellido como apellido_entrenador.

De la tabla wods usamos las columnas tipo como tipo wod y duración.

3. Vista_reservas_por_clase

El objetivo es saber cuántos inscriptos tiene cada clase así nos sirve para controlar cupos. Necesito las tablas clases y reservas, hacer un JOIN y luego agrupar por clase con un GROUP BY; hacer un count de reservas para que me cuente el número de reservas.

Necesito de la tabla clases las columnas: id clase, fecha, hora y tipo clase.

De la tabla reservas uso la columna id reserva para hacer el count.

4. Vista clientes membresias

El objetivo es ver qué membresía tiene cada cliente. Necesitamos las tablas clientes y membresías. Nos sirve para hacer un seguimiento de planes activos.

De la tabla clientes usamos las columnas id_cliente, nombre, apellido, y de la tabla membresías usamos las columnas nombre como nombre_membresia, precio, duración_dias y limite_clases. Hacemos un JOIN de las 2 tablas.

5. Vista_pagos_clientes

El objetivo es ver el historial de pagos. Usamos las tablas clientes, pagos y membresías y hacemos un JOIN.

De la tabla clientes usamos las columnas nombre, apellido.

De la tabla pagos usamos las columnas id_pago, fecha_pago, método_pago.

De la tabla membresías usamos las columnas nombre como nombre_membresia, y la columna precio.

FUNCIONES

1. Calcular la edad de un cliente

Calcula la edad en años a partir de fecha_nacimiento.

2. Cantidad de clases reservadas por un cliente

Nos devuelve cuántas reservas realizó un cliente

TRIGGERS

1. Evitar sobrepasar el cupo máximo de una clase

Nos sirve porque no nos permitirá hacer reservas si la clase ya alcanzó su cupo. Usamos la tabla reservas y el tipo BEFORE INSERT ya que necesitamos que antes de que se inserte un dato en la tabla reservas, o sea antes de que un cliente pueda reservar una clase (antes que se inserte una nueva fila en la tabla reservas), no nos deje hacerlo si el cupo de esa clase está lleno. El FOR EACH ROW es que se ejecuta una vez por cada fila agregada (reserva) que se intenta insertar.

Si el trigger falla, la reserva no se guarda.

Este trigger nos garantiza la integridad del negocio porque evita que se registren más reservas que el cupo permitido así se controla que no se exceda el cupo máximo de una clase antes de registrar una reserva.

2. Activar un cliente automáticamente al registrar un pago

El objetivo de este trigger es cambiar el estado del cliente a Activo cuando realice un pago. Usamos la tabla pagos. Es de tipo AFTER INSERT porque una vez que el cliente realice el pago (se inserte un dato en la tabla pagos), recién cambia su estado de inactivo a activo.

Así activa automáticamente al cliente cuando paga; se actualiza automáticamente el estado del cliente cuando se registra un pago.

STORED PROCEDURES

1. Registrar una reserva de una clase

El objetivo de este stored procedure es registrar una reserva sólo si el cliente está activo y si la clase existe.

El control de cupo ya lo hace el trigger, así que aquí no se repite. Se evita que clientes inactivos se inscriban a las clases, y asegura que la clase a la que se inscriba exista.

2. Registrar un pago de membresía

El objetivo es registrar un pago y activar automáticamente al cliente reforzando así el trigger.

El trigger AFTER INSERT ON pagos era el que se encargaba de activar al cliente. Aquí se registra el pago, inserta el pago y así dispara el trigger que activa automáticamente al cliente una vez que ya se realizó el pago.

Enlace al script SQL:

[CH_MySQL/entregable_n2 at main · EmiTorres7/CH_MySQL](#)